TLR-Raft における高速な分散ログ再構築

永見 拓人[1,a)]　駒野 雄一[1,b)]

概要：Kubernetes に代表される分散システムにおいては、合意アルゴリズム Raft が基盤技術として使われている。Raft を用いると、全てのサーバーにログが完全複製される。そのため、1 つでもサーバーが侵害されればデータが漏洩する恐れがあり、Raft は構造的に機密性が低いという問題を抱えていた。それに対し、我々は CSS2025 および SCIS2026 で、Raft と秘密分散を組み合わせた合意アルゴリズム「Threshold Leak-Resistant Raft Consensus Algorithm （TLR-Raft)」(Partially Leak-Proof Raft より改名）を提案した。しかし、これまでに提案した TLR-Raft は、他ノードが持つ分散ログエントリを用いて失われた分散ログエントリを再構築 （分散ログ再構築） するプロセスが低速であり、多くの処理時間を要するという課題を抱えていた。本稿では、ログ 1 つずつに対しての処理ではなくバルク処理を行うことで、TLR-Raft の分散ログ再構築を高速化する手法を提案する。

キーワード：合意アルゴリズム, Raft, 秘密分散法, 最適化

Faster Reconstruction of Secret-Shared Logs in TLR-Raft

Takuto Nagami[1,a)]　Yuichi Komano[1,b)]

Abstract: The Raft consensus algorithm has been a foundational technology in distributed systems such as Kubernetes. However, because Raft fully replicates logs across all servers, compromising a single server may result in the leakage of all data. Thus, Raft inherently has poor confidentiality guarantees. To address this issue, we proposed the Threshold Leak-Resistant Raft Consensus Algorithm (TLR-Raft), which combines Raft with a secret-sharing scheme, at CSS2025 and SCIS2026. However, TLR-Raft requires reconstructing missing secret-shared log entries from secret-shared logs stored on other servers, and the previously proposed reconstruction process is computationally expensive because it processes each log entry individually. In this thesis, we propose a method to accelerate secret-shared log reconstruction in TLR-Raft by processing multiple log entries in bulk.

Keywords: Consensus Algorithm, Raft, Secret Sharing Scheme, Optimization

1. はじめに

2014 年に Ongaro と Ousterhout によって提案された合意アルゴリズムである Raft[6] は、現在、Kubernetes をはじめとする様々な分散システムにおいて基盤技術として使われている。しかし、Raft は全サーバーにログを完全複製するアルゴリズムであるため、任意のサーバーが窃取されただけで機密情報が漏洩してしまう恐れがあり、構造的に機密性の問題を抱えている。

サーバーへの侵害による情報漏洩を防ぐアプローチとして、AES[5] に代表される共通鍵暗号や RSA 暗号 [7] に代表される公開鍵暗号を用いた暗号化がよく用いられてきた。実際、etcd に代表される Raft を組み込んだプロダクトでは暗号化機能を提供しているが、暗号化を用いてシステムに機密性をもたせるには複雑な鍵管理が必要となり、運用次第で結局漏洩耐性が上がらないケースも数多く存在する。

複雑な鍵管理を必要とせずに情報の機密性を維持するアプローチとして、秘密分散法 [2], [9] がある。その一つであ

[1] 千葉工業大学
Chiba Institute of Technology
[a)] s2232703wf@chibatech.ac.jp
[b)] yuichi.komano@chibatech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

る (k, n) しきい値秘密分散法では、機密情報を n 個のシェアに分割し、n 個のサーバーにシェアを一つずつ分散保存させる。(k, n) しきい値秘密分散法を用いた場合、k 個（しきい値）以上のサーバーがシェアを持ち寄れば機密情報を復元できるが、k 個未満のシェアからは元の機密情報に関する情報を得ることはできない。

我々 [10], [11] は、複雑な鍵管理を用いることなく Raft がもつ構造的な機密性の問題を解決する手法として、(k, n) しきい値秘密分散法を利用した「Threshold Leak-Resistant Raft Consensus Algorithm（TLR-Raft）」（Partially Leak-Proof Raft より改名）を提案した。TLR-Raft は、Raft の派生アルゴリズムであり、同論文で定義した分散システムにおける新たなセキュリティ評価指標である「閾値機密性」（部分的漏洩耐性より改名）を満たしている。

TLR-Raft では、リーダーが機密情報をシェアに分割して各サーバーに分散保存する。しかし、あるサーバーが一時的な断絶によって分散ログを受け取れなかった場合、そのサーバーは既存の分散ログの一部を保持しないままクラスタに復帰することになる。このように、既存の分散ログを保持していないサーバーがクラスタに参加・復帰した場合、リーダーは他のサーバーが保持する分散ログエントリを収集して仮想ステートのログを復元し、当該サーバー向けのシェアを再構築して再送する必要がある。

しかし、これまでに提案した TLR-Raft では、再構築が分散ログエントリ 1 つずつに対して行われており、多数のエントリを再構築する際には大量の処理時間を要するという課題があった。そこで本稿では、ログ 1 つずつに対しての処理ではなくバルク処理を行うことで、TLR-Raft の分散ログ再構築を高速化する手法を提案する。

1.1 本研究の貢献

本研究の貢献は以下のとおりである。

- TLR-Raft の分散ログ再構築が分散ログエントリ 1 つずつに対して行われ、多数のエントリを再構築する際に多くの処理時間を要するという問題を提起する。
- 上記の問題について、リーダーが欠落しているコミット済み分散ログの範囲を一度に特定し、その範囲全体をバルク処理で再構築する手法を提案する。
- 提案した手法によって分散ログ再構築が高速化されることを、実装によって旧手法と比較評価する。

本論文の構成は以下のとおりである。2 節では準備として、合意アルゴリズム Raft と (k, n) しきい値秘密分散法について説明する。次に 3 節では、TLR-Raft の概要と分散ログ再構築の仕組みを説明したのち、その問題点を述べる。その後 4 節では、分散ログ再構築をバルク処理で高速化する手法を提案し、5 節でその手法の有効性を実装評価する。そして 6 節で実験結果について考察し、7 節でまとめを述べる。

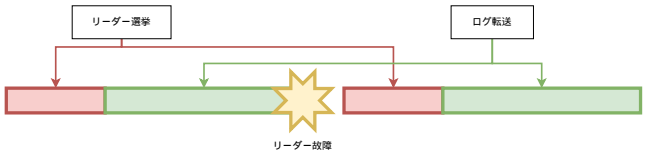


図 1 Raft におけるターム

2. 準備

本節では TLR-Raft の前提知識として、Raft と、TLR-Raft で利用する秘密分散法について説明する。

2.1 Raft

Raft[6] は、複製されたログを分散システム上で管理するために用いられる合意アルゴリズムである。Ongaro と Ousterhout[6] は、同様の目的で作られた Paxos[3], [4] の難解さに課題を見出し、同じ結果・効率をもちながらも、Paxos よりもアルゴリズムが理解しやすく、実用的なシステムを構築するための基盤を提供することを目標として、Raft を提案した。

Raft は、Kubernetes のデータストアとして使われている etcd に組み込まれ、現代の Web/AI インフラの根幹部分を支えている。また、Hyperledger[1] 等のブロックチェーンにおいても、合意形成アルゴリズムとしてしばしば用いられている。

Raft は、Replicated State Machine[8] 機構に則って構成されている。典型的な Replicated State Machine 機構では、分散システムの複数のサーバーが操作履歴（ログ）を保持し、それぞれの State Machine がログを常に同じ順序で計算して同一のステートを算出する。合意アルゴリズムである Raft によりログを管理・複製することで、複数のサーバーで複製されたステートを保持できる。

Raft は、選挙とログ複製の 2 つのフェーズからなる。選挙フェーズでは、リーダー選挙が行われる。ログ複製フェーズでは、リーダーがもつログをすべてのサーバーに複製して一貫した状態に保つための仕組みであるログ複製（Log Replication）が行われる。選挙とログ複製のフェーズを 1 回ずつ合わせたものがタームと呼ばれ、リーダーの交代を境目としてタームを積み重ねていくのが Raft のライフサイクルである（図 1）。

2.1.1 リーダー選挙

Raft は合意を形成するために、特定のリーダーを選出し、そのリーダーに複製されたログの管理に関する全ての責任を与える。リーダーの選出に用いられるアルゴリズムは、リーダー選挙（Leader Election）とよばれる。

図 2 に示すように、Raft においては、各サーバーはリーダー、フォロワー、候補者のいずれかの状態をとる。リーダーは生存確認のため、複製するログが無くても空のログ複製メッセージを各サーバーに定期的に送信している。し

© 2026 Information Processing Society of Japan

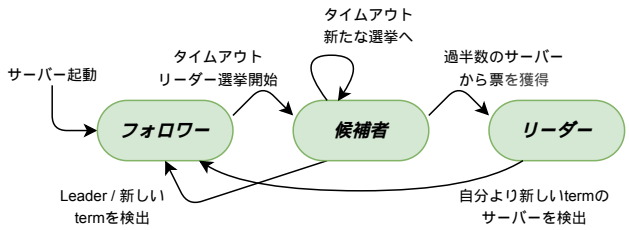


図 2 Raft におけるサーバーの状態遷移

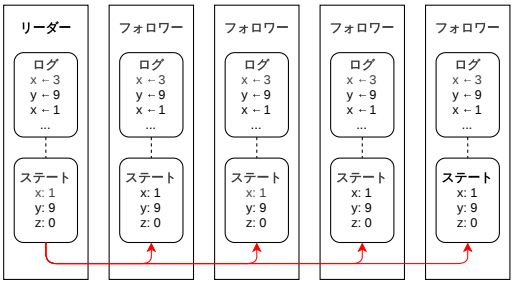


図 3 Raft のデータ配置イメージ

かし、フォロワーが一定期間リーダーからのメッセージを受信できない場合、そのフォロワーはリーダーが不通になったと判断して自身のタームを 1 つ進めて候補者に遷移し、他のすべてのサーバーに投票をリクエストする。候補者は過半数のサーバーから承認を得ると新たなリーダーに昇格する。フォロワーからの承認を得るためには、最低でも承認するフォロワーが持っているログを全て持っていることが必要であり、これによって最も新しいログまで所持しているサーバーがリーダーになることが保障される。

2.1.2 ログ複製

ログ複製は、リーダーが持つログをすべてのサーバーに複製し、一貫した状態を維持する仕組みである。

Raft は強力なリーダーシップモデルを採用しており、ログエントリはリーダーからフォロワーへの一方向のみに流れる。クライアントからのコマンドを受け取ったリーダーは、それを自身のログに追記し、他のサーバーへ AppendEntries RPC を用いて並行して複製する。フォロワーは、自身のログがリーダーのログに追いついていない場合、そのログの index とタームが自身のログと一致することを確認したうえで、リーダーから受け取ったログエントリを自身のログに追記する。一方、自身のログにリーダーのログと不整合なエントリが存在する場合、フォロワーは該当するエントリ以降を削除し、リーダーのログで上書きする。リーダーは、フォロワーのログが自身のログと一致するまでログエントリを遡って送り続け、これによって全サーバーのログが同一の状態に保たれる。

それぞれのログエントリは、リーダーからクラスタ内で過半数のサーバーに複製された時点でコミット（確定）したと見なされ、その時点をもってステートマシンに適用できるようになる。

2.2 (k, n) しきい値秘密分散法

秘密分散法は、データを複数の断片（シェア）に分割し、その一部を知っていても元の情報を復元できないようにする暗号技術であり、Shamir[9] と Blakley[2] によって独立に提唱された。Shamir と Blakley はそれぞれ、データ D を n 個のシェアに分割し、そのうち任意の k 個のシェアが集まれば D を容易に復元できるが、k 個未満のシェアからは D に関する情報が一切得られないようにする、(k, n) しきい値分散法を提案した。

本稿は、Shamir[9] によって提唱された秘密分散法を使用する。この方式は、多項式補間に基づき、数値あるいは数値に変換可能な秘密データ D を分散する。

シェアの生成においては、ディーラーとよばれる秘密データ D を分散する人物が、次数が $k-1$ であり、定数項 a_0 が秘密データ D そのものとなるランダムな多項式 $q(x)$ を選ぶ。そして、多項式 $q(x)$ に $x = i$ を代入することによって n 個のシェア $D_i = q(x_i)$ を生成し、n 人のユーザーに i と D_i を一組ずつ配布する。

シェアから秘密情報を復元する際は、任意の k 個のシェア D_i と、それに対応する i が必要となる。これらの点から多項式補間により元の多項式 $q(x)$ の係数を特定することがで、定数項から秘密データ $a_0 = D$ を復元できる。方式を実装する際には、ラグランジュ補間が使われることが一般的である。

3. TLR-Raft とその問題点

本節では、TLR-Raft[10], [11] の概要と、TLR-Raft の要点である「改良されたリーダー選挙」「分散ログ転送」と「未配送分散ログの再送」について説明する。その後、現在の TLR-Raft が抱える問題点を述べる。

3.1 TLR-Raft

TLR-Raft（Threshold Leak-Resistant Raft Consensus Algorithm）は、(k, n) しきい値秘密分散法を利用して、Raft[6] に閾値機密性を持たせた Raft の派生アルゴリズムである。

Raft においては、図 3 のように、リーダーがログをフォロワーに配布して、各フォロワーにログおよびステートが複製される。TLR-Raft においては、図 4 のような形でデータが配置される。クラスタ全体で保存されているデータは仮想ステート（Virtual State）とよばれ、これを秘密分散で変換したシェアがそれぞれのサーバーがもつ分散ステート（Secret-Shared State）である。それぞれのサーバーは異なる分散ステートをもち、分散ステートの変更履歴である分散ログ（Secret-Shared Log）の整合性を、Raft が保証している安全性に基づいてクラスタ内で保つ仕組み

© 2026 Information Processing Society of Japan

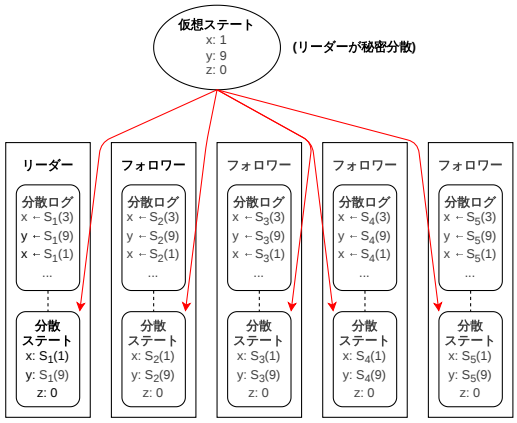


図 4 TLR-Raft のデータ配置イメージ

になっている。

秘密分散の安全性を分散システムに拡張することで、TLR-Raft に対して「データを分散保存しているサーバーのうち、しきい値未満の台数のサーバーが侵害されて分散ステートが奪取されても、本来のデータ（仮想ステート）を復元することができない」という性質を生み出すことができる。我々はこの性質を「閾値機密性」（部分的漏洩耐性より改名）と名付け [10]、TLR-Raft は閾値機密性を持つことを示した。

また、Raft と TLR-Raft アルゴリズムをそれぞれ Go 言語によって実装し簡易実験を行った [10]。その結果、秘密分散によるオーバーヘッドが顕著にあらわれたが、100 バイトのエントリに対して 1 ミリ秒に満たないほどのオーバーヘッドであり、実際の通信遅延を考慮した際にはかなり実用的な値が出ていると考えられる。

3.2 改良されたリーダー選挙

TLR-Raft は、Raft と同様のリーダー選挙を基本としているが、2 つの変更 [11] を加えたものを用いる。

候補者の承認には、改良された RequestVote RPC が用いられる。この RequestVote RPC では、候補者が保持する分散ログのうち最後にコミットされたエントリの index とタームを送信し、レシーバーは候補者のコミット済み分散ログが自身の保持するコミット済み分散ログに追いついていれば承認する。Raft の元々の RequestVote RPC では候補者が保持する最新のログエントリに基づいて承認が行われていたが、TLR-Raft ではコミットされていない分散ログエントリは後述の理由により保持されないため、コミット済みの分散ログに基づく承認で十分である。

また、候補者はリーダーに昇格する際、自身が保持する分散ログのうちコミットされていないエントリを削除する。コミットされていないエントリは、十分な数のサーバーにシェアが分散されていない可能性があり、リーダーが遷移した場合に分散ログ再構築が行えないおそれがある。このようなエントリを残したまま分散ログ転送を行うと不整合が生じるため、昇格時に削除する。これにより、リーダーが保持する分散ログはコミット済みの分散ログエントリのみとなり、昇格後の分散ログ転送が正常に動作する。

3.3 分散ログ転送

分散ログ転送（Secret-Shared Log Transmission）は Raft におけるログ転送に対応する概念であり、機密情報から生成された秘密分散のシェアをそれぞれのサーバーに転送するための仕組みである。

まず、Raft と同様に、ステートに対する変更は全てリーダーへと送られる。リーダーはこれを仮想ステートのログとして扱い、このログが正しく配送されるまで自身の揮発性ステートに一時保存する責任を負う。

リーダーは、追加するログエントリに対し、(k, n) 秘密分散（k はあらかじめ設定された値、n はサーバーの台数）のディーラーとしてサーバー数分のシェアを生成する。シェアの生成時に多項式に代入される i は、重複を防ぐためユニークにサーバーに割り振られた ID から決める。これによって、同じログエントリから同じサーバーに対して発行されるシェアは常に同じであることが保障される。

最終的にリーダーは、各フォロワーに対して、生成したシェアを分散ログのエントリとして送信する。分散ログエントリは、クラスタ内の過半数と $k+1$ のうち大きい方の台数に相当するサーバーに転送されたことが確認できた時点でコミット（確定）される。このエントリがコミットされた瞬間、リーダーが一時保存していた仮想ステートの該当ログエントリを可及的速やかに消去する。コミットされたエントリは、リーダーが仮想ステートのログを消去した後でも、他のサーバーが保持する分散ログエントリから再構築することが可能である。

3.4 未配送分散ログの再構築

Raft において、リーダーが保存していてフォロワーが保存していないログは都度再送される。Raft ではリーダーが自分で保存しているログをそのまま再送すればよいが、TLR-Raft ではタイミングによって以下の 2 パターンに処理が分かれる。

コミット前 コミット前は、リーダーが仮想ステートのログを自身の揮発性ステートに一時保存しているため、そこからシェアを再構築して再送する。

コミット後 コミット後は、仮想ステートのログを直接保存しているサーバーが存在しないため、リーダーが以下の手順を踏んでシェアを再構築する必要がある。

まず、リーダーは仮想ステートのログを復元するために、該当のフォロワーを除くすべてのフォロワーの FetchLog RPC を呼び出す。FetchLog RPC はこのために生み出された新しい RPC 定義である。リーダーはこれが早く帰っ

© 2026 Information Processing Society of Japan

てきた順に値を読み取り、自分が持っている分散ログエントリと合わせて k 個以上のシェアが集まった段階で仮想ステートのログエントリを復元し、再度シェアを生成して配布する。

FetchLog RPC

引数

- *term*: リーダーのターム
- *index*: 欲しい分散ログの index

結果

- *term*: 呼び出し先フォロワーのターム。リーダーが新しいタームを検知する用途
- *success*: 該当する分散ログエントリが存在すれば true
- *entry*: 該当する分散ログエントリ

レシーバーの実装

(1) $term < currentTerm$ なら $success = false$ と空の *entry* で応答
(2) 該当するエントリが自分の元に存在しなければ、$success = false$ と空の *entry* で応答
(3) 該当するエントリを取得して *entry* にセットし、$success = true$ で応答

リーダーはこの手順によって再構築されたシェアを、AppendEntries RPC に対して true が返ってくるまで自身の揮発性ステートに一時保存する責任を負う。AppendEntries RPC で true が返ってきて分散ログが正確に転送されたことが確認できたら、リーダーが一時保存していた再計算済みシェアは、可及的速やかに消去される必要がある。

3.5 TLR-Raft の問題点

前節で説明したとおり、TLR-Raft の分散ログ再構築は、フォロワーがコミット済みの分散ログに追いついていない場合に、不足している分散ログエントリを復元するために行われる。しかし、これまでに提案した TLR-Raft では、この分散ログ再構築が分散ログエントリ 1 つずつに対して行われている。

具体的には、リーダーがフォロワーに分散ログエントリを送信する際、フォロワーがコミット済みの分散ログに追いついていない場合、リーダーは不足している分散ログエントリの index を 1 つずつ特定し、各 index に対して FetchLog RPC を送信して k 個のシェアが集まるのを待ち、シェアを再構築していた。すなわち、m 個の分散ログエントリが不足している場合、FetchLog RPC による再構築処理が m 回連続して行われることになる。

多数の分散ログエントリが欠落する状況は、例えばフォロワーが長期間停止していた場合や、フォロワーの分散ログがリセットされた場合に発生する。このような状況では、再構築対象のエントリ数に比例した大量の RPC が発生し、分散ログ転送により大きな処理時間ならびにネットワーク遅延を課しているという課題がある。

4. 提案手法

本研究では、3.5 節で述べた課題を解決するため、TLR-Raft における分散ログ再構築の高速化手法を提案する。具体的には、ログ 1 つずつに対しての処理ではなく、バルク処理を行うことで高速化を図る。

4.1 分散ログ転送の流れの変更

本節では、分散ログ転送の流れを変更し、分散ログ再構築をバルク処理で行う手法を説明する。

旧来の方式では、リーダーがフォロワーに分散ログエントリを送信する際、フォロワーがコミット済みの分散ログに追いついていない場合、リーダーは不足しているエントリの index を 1 つずつ特定し、各 index に対して FetchLog RPC を送信して再構築を行っていた（3.4 節）。そのため、分散ログ再構築に必要な FetchLog RPC の回数は、欠落している分散ログエントリの数に比例していた。

一方、提案手法では、リーダーはまず AppendEntries RPC の送受信によってフォロワーの分散ログの状態を確認し、不足しているコミット済み分散ログの範囲を一度に特定する。リーダーは各フォロワーに対して次に送信すべき index（nextIndex）を管理しているため、あえて entry が空の状態で AppendEntries RPC を繰り返し送り、prevLogIndex および prevLogTerm の一致確認と nextIndex の調整によって、フォロワーがコミット済み分散ログのどの index までを保持しているかを特定する。フォロワーがコミット済み分散ログに追いついていない場合（nextIndex が commitIndex 以下である場合）、リーダーは、不足しているコミット済み分散ログの範囲を $[fromIndex, toIndex]$ として特定し、その範囲全体を対象とした分散ログ再構築を一括して行う。

範囲を対象とした分散ログ再構築は、以下の手順で行われる。まず、リーダーは該当のフォロワーを除くすべてのフォロワーに対して、$fromIndex$ から $toIndex$ までの範囲を指定した FetchLogs RPC（4.2 節）を送信する。次に、リーダーは返ってきた分散ログエントリを、自分が保持する分散ログエントリと合わせて、各 index について k 個以上のシェアが集まった時点で、範囲内のすべての仮想ステートのログエントリを復元する。最後に、リーダーは復元した仮想ステートのログエントリから、該当のフォロワー向けのシェアをまとめて再構築し、揮発性ステートに一時保存する。再構築したシェアは、次回以降の AppendEntries RPC で一括して送信され、フォロワーがコミット済み分散ログに追いついたことが確認できた時点で、可及的速やかに消去される。

© 2026 Information Processing Society of Japan

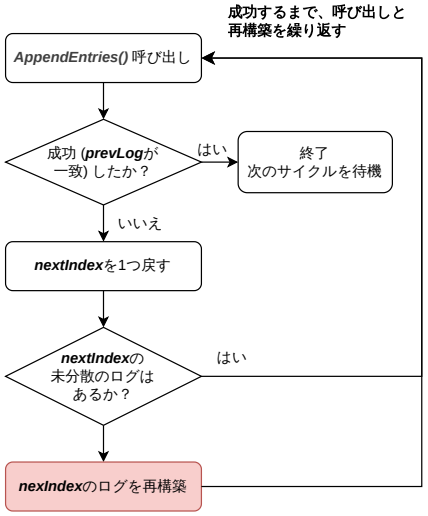


図 5　旧方式における分散ログ再構築の流れ

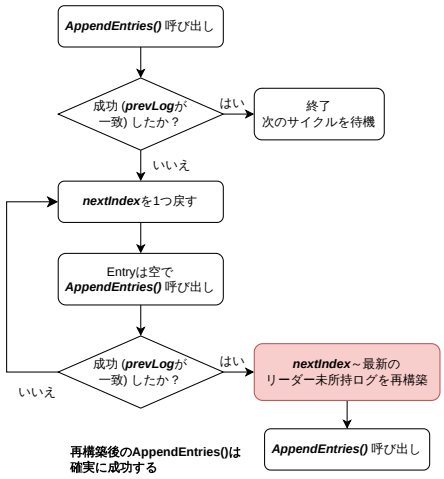


図 6　新方式における分散ログ再構築の流れ

図 5 および図 6 に、旧方式と新方式における分散ログ再構築の流れを示す。旧方式では、欠落している分散ログエントリの index が見つかるたびに FetchLog RPC を送信してシェアを収集・再構築し、AppendEntries RPC での転送を欠落エントリの数だけ繰り返す（図 5）。一方、新方式では、まず欠落しているコミット済み分散ログの範囲を一度に特定し、その範囲をバルク処理で再構築したうえで、AppendEntries RPC により一括して転送する（図 6）。

ここで、提案手法によって分散ログの抜け漏れが発生しないことを確認する。Raft の Log Matching Property により、コミットされた分散ログエントリの index は 1 から $commitIndex$ まで連続しており、フォロワーが不足しているコミット済みのエントリは必ずこの範囲に含まれる。また、コミット規則（3.3 節）により、コミットされたエントリは過半数と $k+1$ のうち大きい方の台数以上のサーバーに転送されている。したがって、リーダー自身と該当のフォロワーを除いても $k-1$ 個以上のシェアが他のフォロワーに存在し、リーダー自身のシェアと合わせて k 個以上のシェアを必ず収集できる。以上より、リーダーが特定した範囲内のすべてのエントリについて分散ログ再構築が可能であり、分散ログの抜け漏れは発生しない。

4.2　FetchLog RPC の変更

4.1 節で述べた分散ログ転送の流れの変更に伴い、FetchLog RPC の仕様も変更する必要がある。旧来の FetchLog RPC（3.4 節）は、1 つの index を指定して 1 つの分散ログエントリを取得するものであった。しかし、バルク処理による分散ログ再構築を行うためには、リーダーが一度の RPC で範囲内の複数の分散ログエントリを取得できる必要がある。そこで、FetchLog RPC を、index の範囲を指定して複数の分散ログエントリを取得する「FetchLogs RPC」で置き換える。

FetchLogs RPC

引数

- $fromIndex$: 取得したい分散ログエントリの範囲の開始 index
- $toIndex$: 取得したい分散ログエントリの範囲の終了 index

結果

- $success$: 該当する分散ログエントリを取得できれば true
- $entries$: 取得した分散ログエントリのリスト

レシーバーの実装

(1) $fromIndex$ から $toIndex$ までの範囲の分散ログエントリを自身の分散ログから取得し、$entries$ にセットして $success = true$ で応答

(2) 該当するエントリが自分の元に存在しなければ、空の $entries$ と $success = false$ で応答

なお、旧来の FetchLog RPC の結果に含まれていた term（呼び出し先のターム）は、変更後の FetchLog RPC では省略している。タームの新しさの検知は、分散ログ転送の本体である AppendEntries RPC の応答によって行うことができるため、FetchLog RPC の結果に term を含める必要はない。

5.　実験評価

本節では、4 節で提案したバルク処理方式の有効性を評価する。評価では、フォロワー 1 台の分散ログをリセットし、そのフォロワーがクラスタ内のコミット済み分散ログに追いつくまでに要した時間を、分散ログを 1 つずつ再構

© 2026 Information Processing Society of Japan

築する旧手法と比較した。

5.1 評価環境

旧手法（3.4 節）と新手法（4 節）をそれぞれ Go 言語で実装した。文献 [10] の評価と同様に、通信のばらつきによる影響を抑えるため、1 つの実行ファイルから複数のゴルーチンを立ち上げてサーバーとみなし、プロセス間通信（Go の channel）を経由して RPC 呼び出しを行う形とした。

評価は、Apple Silicon（arm64）を搭載した MacBook Air 上で動作する UTM（QEMU ベースの仮想マシン）により構築した仮想マシン上で行った。仮想マシンには Ubuntu 24.04 LTS（arm64）を導入し、CPU 6 コア・メモリ 12GB を割り当てた。Go のバージョンは 1.27.0 を使用した。

実験では、n 台のサーバー、ハートビートの感覚は 100ms でクラスタを構成し、リーダーに対して d_1 バイトのペイロードを持つ d_2 個の書き込みコマンドを発行して、コミット済み分散ログを蓄積した。その後、フォロワー 1 台の分散ログをリセットし、そのフォロワーがクラスタ内のコミット済み分散ログに追いつくまでに要した時間を計測した。計測には Go のベンチマークテスト機能を用い、各条件について 5 回のベンチマークを行って平均時間と分散を算出した。パラメータは、$(n, k) \in \{(3,2),(5,2),(5,3),(5,4),(7,2),(7,4),(7,6)\}$、$d_1 \in \{1000, 10000, 100000\}$ バイト、$d_2 \in \{100, 1000, 10000\}$ とした。

5.2 評価結果

分散ログ再構築の性能を評価するため、フォロワー 1 台の分散ログを全てリセットし、そのフォロワーがクラスタ内のコミット済み分散ログに追いつくまでに要した時間を計測した。表 1 に、旧手法と新手法について、条件（n はサーバー台数、k は秘密分散のしきい値、d_1 はペイロードサイズ、d_2 は書き込みコマンド数）ごとの平均時間（ms）と分散（ms^2）、および高速化の倍率（旧手法の平均時間 ÷ 新手法の平均時間。大きいほど新手法が高速）を示す。

表 1 から、$d_2 = 10000$ のように再構築する分散ログエントリ数が多い条件では、新手法が旧手法と比較して最大で約 37 倍の高速化を達成した（$n = 7$, $k = 2$, $d_1 = 1000$, $d_2 = 10000$）。$d_1 = 1000$ のとき、新手法の平均時間は d_2 の値によらず約 194ms でほぼ一定であるのに対し、旧手法の平均時間は d_2 の増加に伴って大きく増加し、$d_2 = 10000$ では約 6200ms に達した。一方、$d_2 = 100$ のように再構築エントリ数が少ない条件では、倍率が約 0.5 となり、新手法が旧手法の約 2 倍の時間を要した。

表 1 分散ログ再構築にかかった時間の比較

n	k	d_1	d_2	旧手法		新手法		倍率
				平均 (ms)	分散 (ms^2)	平均 (ms)	分散 (ms^2)	(旧/新)
3	2	10^3	10^2	99.588	0.11	193.888	13.90	0.51
			10^3	102.193	10.32	195.179	12.21	0.52
			10^4	6232.209	30658.83	194.046	3.62	32.12
		10^4	10^2	98.966	0.32	198.548	3.08	0.50
			10^3	105.388	3.99	196.122	8.88	0.54
			10^4	6260.291	24821.14	374.524	474.36	16.72
		10^5	10^2	98.519	0.06	193.892	9.65	0.51
			10^3	285.808	67.21	291.952	79.24	0.98
			10^4	11173.308	6418941.61	1815.857	12349.45	6.15
5	2	10^3	10^2	99.301	0.67	198.011	11.81	0.50
			10^3	103.126	12.07	194.509	8.14	0.53
			10^4	6162.066	1029.30	194.367	11.82	31.70
		10^4	10^2	99.466	0.11	194.439	9.49	0.51
			10^3	103.893	4.06	195.918	5.88	0.53
			10^4	6514.803	146220.42	393.937	77.58	16.54
		10^5	10^2	99.364	1.20	196.156	20.03	0.51
			10^3	302.612	62.41	289.204	64.52	1.05
			10^4	13487.366	983473.25	2677.034	494801.82	5.04
	3	10^3	10^2	99.136	0.25	197.043	15.10	0.50
			10^3	116.581	112.84	198.251	28.24	0.59
			10^4	6292.994	3814.18	196.219	13.49	32.07
		10^4	10^2	99.026	0.29	197.382	9.06	0.50
			10^3	116.224	34.37	193.476	3.47	0.60
			10^4	6903.738	246318.64	491.402	186.73	14.05
		10^5	10^2	99.512	0.13	195.036	3.95	0.51
			10^3	370.356	265.47	384.754	88.68	0.96
			10^4	19341.669	9223372.04	3469.559	811536.28	5.57
	4	10^3	10^2	98.977	0.10	193.292	13.46	0.51
			10^3	110.614	26.53	194.819	10.40	0.57
			10^4	6474.193	32611.64	195.392	17.63	33.13
		10^4	10^2	99.486	0.33	195.373	3.34	0.51
			10^3	184.430	137.56	196.003	12.08	0.94
			10^4	6701.041	46056.31	533.747	665.00	12.55
		10^5	10^2	101.143	3.02	193.828	2.05	0.52
			10^3	427.072	308.91	452.267	274.49	0.94
			10^4	19017.486	9223372.04	3836.933	537694.50	4.96
7	2	10^3	10^2	99.790	0.34	195.421	20.25	0.51
			10^3	116.182	312.73	197.226	26.75	0.59
			10^4	7308.119	462150.36	195.367	5.46	37.41
		10^4	10^2	99.837	0.09	195.871	52.18	0.51
			10^3	106.749	29.30	192.932	1.51	0.55
			10^4	6618.755	111585.10	388.987	64.08	17.02
		10^5	10^2	99.320	2.06	195.697	23.21	0.51
			10^3	295.733	0.98	286.840	45.55	1.03
			10^4	21358.634	2219248.96	10495.848	9223372.04	2.03
	4	10^3	10^2	99.484	0.67	196.521	13.35	0.51
			10^3	107.430	25.40	193.076	2.18	0.56
			10^4	6473.649	1479.42	199.203	87.02	32.50
		10^4	10^2	99.045	0.27	198.171	3.76	0.50
			10^3	194.332	73.82	194.530	4.48	1.00
			10^4	6676.360	8274.75	562.309	401.39	11.87
		10^5	10^2	98.493	0.06	193.872	6.78	0.51
			10^3	411.230	129.41	477.496	258.59	0.86
			10^4	24430.674	6014753.44	8228.338	9223372.04	2.97
	6	10^3	10^2	100.623	0.25	195.335	10.11	0.52
			10^3	146.217	1941.28	192.267	0.54	0.76
			10^4	6399.509	3929.26	288.378	73.41	22.19
		10^4	10^2	98.886	0.26	194.990	2.32	0.51
			10^3	200.338	24.46	196.625	7.28	1.02
			10^4	6821.853	3225.48	680.251	874.95	10.03
		10^5	10^2	101.863	4.16	197.202	5.56	0.52
			10^3	592.530	102.81	562.732	277.41	1.05
			10^4	36736.101	11719222.70	15474.951	9223372.04	2.37

6. 考察

本節では、5.2 節の実験結果について考察する。

まず、前述したとおり、再構築する分散ログエントリ数が多い条件（$d_2 = 10000$）では、新手法が旧手法を大きく上回った。旧手法は再構築エントリの index ごとに FetchLog RPC を送信して再構築するため、処理時間が再構築対象のエントリ数に比例する。一方、新手法は欠落範囲を一度に特定してバルク処理で再構築するため、再構築に必要な RPC の回数がエントリ数に依存しない。この結果は提案手法の原理に沿うものであり、想定どおりの結果といえる。

一方、再構築エントリ数が少ない条件（$d_2 = 100, 1000$）では、新手法が旧手法よりも遅い結果となった。これは、新手法の処理時間が実装上の制約により約 190ms を下回ら

© 2026 Information Processing Society of Japan

ないことが主因である。新手法ではリーダーの処理がハートビートの間隔に同期しており、分散ログ再構築に移行する際に 1 回のハートビート周期をスキップするため、再構築対象のエントリ数が少ない場合でも約 190ms の時間を要する。一方、旧手法はエントリ数が少なければより短い時間で再構築を完了できる。したがって、再構築対象のエントリ数が少ない状況では旧手法が有利となるが、どちらにしても数百ミリ秒程度の時間で再構築が完了するため、実用上は大きな問題にはならないと考えられる。

また、ペイロードサイズ d_1 が小さいほど高速化の効果が顕著であることも確認できた。$d_2 = 10000$ の条件に着目すると、$d_1 = 1000$ のときは〜32 倍、$d_1 = 10000$ のときは〜17 倍、$d_1 = 100000$ のときは〜6 倍の高速化となっている。これは、ペイロードが大きいほど秘密分散の計算オーバーヘッドが大きくなり、そのオーバーヘッドが旧手法・新手法の双方に共通してかかるため、両者の相対的な差が縮まるためであると考えられる。

最後に、実験の実行方法の観点から考察する。本実験では、すべてのノードを同一プロセス内のゴルーチンとして起動し、Go の channel によるプロセス間通信で RPC を模擬した。このため、ネットワーク遅延はほぼゼロであり、通信の往復時間が支配的でない環境での結果である。それにもかかわらず、$d_2 = 10000$ の条件で大きな差が現れたことは、新手法の効果が我々の想定を上回っていることの表れである。ネットワーク遅延が大きな実際の分散環境では、RPC の往復回数の差が処理時間に加算されていくため、新手法の優位性はより一層際立つと考えられる。

7. まとめ

本稿では、TLR-Raft の分散ログ再構築を高速化する手法を提案した。TLR-Raft では、フォロワーがコミット済みの分散ログに追いついていない場合、リーダーが他のサーバーの分散ログエントリから仮想ステートのログを復元し、シェアを再構築して再送する必要がある。これまでに提案した TLR-Raft では、この再構築が分散ログエントリ 1 つずつに対して行われていたため、多数のエントリを再構築する際に大量の処理時間を要するという課題があった。

本稿では、リーダーが AppendEntries RPC によってフォロワーの欠落しているコミット済み分散ログの範囲を一度に特定し、その範囲全体のシェアを一括して収集・再構築するバルク処理方式を提案した。これにより、分散ログ再構築に必要な FetchLog RPC の回数を、再構築エントリ数に依存しない定数回に削減できる。また、コミットされた分散ログエントリには常に k 個以上のシェアがクラスタ内に存在することを示し、提案手法で分散ログの抜け漏れが発生しないことを確認した。

実装評価の結果、再構築する分散ログエントリ数が多い条件では、新手法は旧手法と比較して最大で約 33 倍の高速化を達成し、提案手法の有効性を確認した。一方、再構築エントリ数が少ない条件では、新手法の処理時間がハートビート周期に起因する実装上の下限（約 190ms）に律速され、旧手法よりも時間を要する場合があることが確認された。これはアルゴリズムの原理による差ではなく実装上の制約に起因するものであり、改善の余地がある。

今後の課題としては、より大規模なクラスタや実際のネットワーク環境を模した環境での評価、およびさらなる最適化の検討などが挙げられる。

謝辞: 本研究は JSPS 科研費 JP24K14951 の助成を受けたものです。

参考文献

[1] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Stathakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery.

[2] G. R. BLAKLEY. Safeguarding cryptographic keys. In *1979 International Workshop on Managing Requirements Knowledge (MARK)*, pages 313–318, 1979.

[3] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, July 1978.

[4] Leslie Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, May 1998.

[5] National Institute of Standards, Technology (NIST), Morris J. Dworkin, Elaine Barker, James Nechvatal, James Foti, Lawrence E. Bassham, E. Roback, and James Dray Jr. Advanced encryption standard (AES), 2001-11-26 00:11:00 2001.

[6] Diego Ongaro and John K. Ousterhout. In search of an understandable consensus algorithm. In Garth Gibson and Nickolai Zeldovich, editors, *Proceedings of the 2014 USENIX Annual Technical Conference, USENIX ATC 2014, Philadelphia, PA, USA, June 19-20, 2014*, pages 305–319. USENIX Association, 2014.

[7] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, February 1978.

[8] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Comput. Surv.*, 22(4):299–319, December 1990.

[9] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.

[10] 永見拓人, 駒野雄一. PLP-Raft：部分的漏洩耐性を持つ合意アルゴリズム. **コンピュータセキュリティシンポジウム** *(CSS)2025, 3B2-3*, 2025.

[11] 永見拓人, 駒野雄一. PLP-Raft における頑健なリーダー選挙アルゴリズム. **暗号と情報セキュリティシンポジウム** *(SCIS)2026, 2G2-3*, 2026.

© 2026 Information Processing Society of Japan